

Progressive Semantic Mechanization:

Compiling Intellectual Work into Deterministic Programs with Typed Semantic Holes

Ivo Matijašević
Independent Researcher
ivomatijasevic@gmail.com

October 2026

Abstract

Which intellectual work should be an executor call (a call to a model or a person)? We model work as a program with typed *semantic holes*. Each site is open or closed separately for decision, production, and acceptance, and separately per input scope. *Progressive semantic mechanization* moves work to deterministic mechanisms through an acceptance-controlled promotion. Under the stated independence assumptions, a share c of production routed to mechanisms gives the first-order horizon ceiling $H_{\text{raw}}/((1-c)\lambda_{\text{st}} + c\lambda_{\text{pc}})$, where H_{raw} is the ungated horizon, λ_{st} the shared blind-spot mass of Part I, and λ_{pc} premature-closure escape. Closure raises this ceiling only where $\lambda_{\text{pc}} < \lambda_{\text{st}}$. Because pointwise evidence cannot separate twins (mechanisms with the same evidence record), a confidence bound valid over the mechanism class can fall below a twin’s fault mass only with the allowed error probability. Same-family evidence inherits the family’s indistinguishability, not a general lower bound on the mechanism’s error. Copies of outputs accepted by sound same-family gates inherit the floor in expectation; the corresponding claim for generalizing builders is an untested hypothesis. Further results give a validation-inclusive break-even volume, a composition bound under completeness and provenance assumptions, and two separate scale gains: from fault-free, rejection-free closure, and from bounded load under an untested hazard hypothesis. The results are elementary statements in the model, not universal laws. No modeled quantity is estimated from data. The simulation uses chosen parameters, the plan is a design, and the instance reports counts only.

1 Introduction

Parts I–III study open production under gates. Part I [1] gives a constant-hazard horizon ceiling of about $H_{\text{raw}}/\lambda_{\text{st}}$ from a shared blind spot. Part II [2] explains a family floor through a representation model: gates decided within one family inherit its stealth set. This holds for sound gates and, under a finite uniform likelihood-ratio bound, up to a tolerance for bounded false rejection. Part III [3] studies escape, cost, and correlation at that floor. Here we ask which work should remain an executor call.

An analogy motivates the question. In the common agent design, a language model does delegated work by fresh judgment at each step, as a clerk once did office work by hand. Yates traces how firms made office work systematic, with standard forms, records, and procedures [12]. The analogy we propose is that systematic records and procedures are what make work mechanizable, and that judgment remains for the cases that no procedure covers. We ask whether model work admits the same path: formalize what can be formalized, mechanize it, and keep an executor for

© 2026 Ivo Matijašević. This work is licensed under a Creative Commons Attribution 4.0 International (CC BY 4.0) license, <https://creativecommons.org/licenses/by/4.0/>.

the residual. On this view an executor, model or person, stays necessary wherever semantics remain open, but it is an ingredient of such a system, not its target.

We treat model inference as residual computation. A work program delegates unresolved work through typed holes and assigns explicit semantics to deterministic mechanisms. Promotion moves the boundary; regression can move it back. We call this *progressive semantic mechanization* (PSM). We have not measured how much work can close.

The ingredients are established: programs with holes, residual programs, compiled reasoning, trusted checking, and promotion of recurring model work into code, which has run in production [9] (§9). We claim no novelty for these. Our addition is the link to the verification floor. Closure needs evidence, and that evidence has limits distinct from the faults of the mechanism it judges.

The model in §2 supports five results in §3: a partial-closure horizon ceiling, limits on promotion evidence, a break-even calculation, a composition bound, and two scale comparisons. Propositions are proved under stated assumptions. Hypotheses 1 and 2 are empirical claims used but not tested. Section 3.6 keeps the remaining questions open. The simulation illustrates results; the plan specifies a design; the instance supplies counts. None validates the model against a real system.

2 Model

Work has constant fault hazard λ and checkpointed segments of length s . An executor produces a bad segment with probability $p = 1 - e^{-\lambda s}$. The ungated horizon is $H_{\text{raw}} = \ln 2/\lambda$. The gate family’s stealth mass $\lambda_{\text{st}} = \lambda_{\text{red}} + \lambda_{\text{irr}}$ is the fraction of bad segments every gate accepts [1]. A model family F has representation r_F , indistinguishability relation $\sim_F$, stealth set Σ_F , and stealth mass $\lambda_F = \mu(\Sigma_F)$ under fault distribution μ [2].

2.1 Work programs and intent

Definition 1 (Work program). A *work program* is a finite directed acyclic graph $W = (V, E)$. Each node $v \in V$ is a *site*: a typed operation with an input space X_v and an output space Y_v . An edge (u, v) means that v consumes an output of u . A *run* executes the sites in an order that is consistent with E . A run of horizon T consists of $n = T/s$ segments, and one site serves each segment. The *volume share* w_v of a site is the fraction of segments that it serves; $\sum_v w_v = 1$.

Definition 2 (Intent predicate and operating distribution). Each site carries a ground-truth predicate $\varphi_v : X_v \times Y_v \rightarrow \{0, 1\}$. The value $\varphi_v(x, y) = 1$ means that y is an acceptable result for the input x . Inputs arrive from an *operating distribution* $\mathcal{D}_v$ on X_v .

We do not assume that φ_v is computable or written down. We drop the index where unambiguous.

2.2 Three closure axes

The axes are *decision* (which transition or local intent), *production* (the artifact), and *acceptance* (the verdict on (x, y)).

Definition 3 (Mechanism and executor). A *mechanism* is a deterministic, terminating procedure whose behaviour is fixed by its text and its explicit inputs. It consults no model and no person at run time. The criterion is explicit, accepted semantics, not determinism: a greedy-decoded model is deterministic, but its semantics are not explicit. An *executor* is a model or a person that serves an axis at run time and whose result is not determined by explicit, accepted semantics.

Definition 4 (Closure state). Let v be a site, $a \in \{\text{dec}, \text{prod}, \text{acc}\}$ an axis, and $S \subseteq X_v$ an input scope. The *closure state* $\kappa_a(v, S)$ takes one of three values.

- CLOSED: a mechanism is *authoritative* for axis a on S . For every input in S the result of the mechanism is used, and no executor serves that axis.
- OPEN: an executor serves axis a on S , and a reviewed determination states that the accepted explicit semantics do not determine the result.
- UNKNOWN: an executor serves axis a on S , and no determination has been made.

The three states apply to an axis that is *served*: an executor or a mechanism serves it. An axis that nothing serves on S is *blocked*. Blocked is not a closure state; it is recorded beside the closure states.

Closure records authority, not correctness or computational simplicity. The state UNKNOWN marks unexamined work.

production	acceptance	meaning
closed	closed	mechanized production and acceptance
open	closed	executor synthesis under a deterministic checker
closed	open	mechanical production judged by an executor
open	open	the largest residual uncertainty

Table 1: Production and acceptance closure. Parts I–III study open production under closed or open gates.

Remark 1 (Relation to checker strength). The fifth open problem of Part III grades a check by whether its property is decidable now, corroborable soon, or gradeable only later. One reading in the present model is that “decidable now” is closed acceptance, and that the other two grades are open acceptance with different delays before grounded evidence arrives. This is a reading, not a result. We do not develop the grading here, and the problem of Part III stays open.

2.3 Semantic holes

Definition 5 (Semantic hole). A *semantic hole* is a typed point of a work program at which an axis that is not closed is delegated to an executor. It specifies an input contract, an output contract, a scope, an authority, the permitted effects, an acceptance, and a provenance requirement.

The containing program owns the contract and transitions. The executor proposes; it does not accept. Promotion preserves the contract while replacing the implementation. A hole’s acceptance is a gate stack in the sense of Parts I–III. *Typed* means only that a hole carries declared contracts over X_v and Y_v and the fields above; we develop no type system. *Compiling* names the build and promotion of a mechanism for a hole; no automatic translation is claimed.

2.4 Residual and frontier

Definition 6 (Closed share, residual, frontier). For an axis a , let $S_v^a \subseteq X_v$ be the union of the scopes on which $\kappa_a(v, \cdot)$ is CLOSED. The *closed share* of the axis is

$$c_a = \sum_{v \in V} w_v \mathcal{D}_v(S_v^a).$$

The *semantic residual* of the axis is the set of pairs $(v, X_v \setminus S_v^a)$ with non-empty scope; its mass is $1 - c_a$. The *unknown share* $u_a \leq 1 - c_a$ is the mass of the residual in state UNKNOWN. The *semantic frontier* is the boundary between the closed scopes and the residual.

In the quantitative results (§§3.1–6), c is the *routing share*: the volume share of production segments that are first sent to a mechanism. When the retained acceptance rejects a mass b of those outputs, the executor serves them, so $c_{\text{prod}} = c(1 - b)$; the two coincide when $b = 0$. Volume weighting matches the horizon quantity. The objective is the smallest residual that correctness, adaptability, and total cost allow. We quantify correctness and cost, but not adaptability.

2.5 Premature closure

Definition 7 (Premature-closure fault and escape). Let the inputs of a scope S be routed to a mechanism $M : S \rightarrow Y$ for production. Its *premature-closure fault set* and *fault mass* are

$$P(M) = \{x \in S : \varphi(x, M(x)) = 0\}, \quad \pi(M) = \mathcal{D}(P(M) \mid S).$$

Let G be the acceptance stack that stays in place behind the mechanism (G can be empty). The stack returns one verdict per pair; for a stochastic gate we take its deterministic reduction, as in Part II. The *escape set* is $E(M, G) = \{x \in P(M) : G \text{ accepts } (x, M(x))\}$, the *escape mass* is $\varepsilon_c = \mathcal{D}(E(M, G) \mid S)$, and the *detected mass* is $d = \pi(M) - \varepsilon_c$. The *premature-closure escape* is the escape mass in units of the raw fault probability of the executor,

$$\lambda_{\text{pc}} = \varepsilon_c / p.$$

All masses are conditional on the routing scope S . If the retained stack rejects nothing, M is authoritative on S in the sense of Definition 4. Both $p\lambda_{\text{pc}}$ and $p\lambda_{\text{st}}$ are segment escape probabilities. Unlike λ_{st} , λ_{pc} can exceed 1. It depends on s through p , whereas ε_c does not. A mechanism fails on a fixed set of inputs. Retrying it reproduces the fault. Rejected outputs therefore need a different implementation, the *fallback*; below this is the open path. These are properties of the deployed mechanism, not averages over builds.

Premature closure also affects acceptance and decision. A closed checker can accept conformance to a contract that misses the intent. Such Goodhart faults contribute to λ_{st} when every gate of the relevant family accepts them; they belong to λ_{irr} only if every available family is blind. False rejection costs a retry, not an escape. A wrong closed decision selects a wrong branch. Our quantitative results concern production.

2.6 Promotion and regression

Definition 8 (Promotion). A *promotion* changes $\kappa_a(v, S)$ from OPEN or UNKNOWN to CLOSED for one mechanism M , one axis a , and one scope S . A *promotion predicate* Π decides it: Π maps a body of *promotion evidence* about the mechanism to a verdict in $\{\text{promote, hold}\}$.

Promotion is a gate whose candidates are mechanisms. Before promotion a candidate can run in *shadow*: its outputs are recorded while the executor stays authoritative. Following Part II, we classify evidence by its deciding substrate, not its author.

Definition 9 (Classes of promotion evidence). Let F be the family of the executor that the mechanism replaces.

- **Same-family evidence.** The verdict on a pair $(x, M(x))$ is decided on the representation of F . Two kinds are in this class: *agreement* of the result $M(x)$ with the output of the executor that the mechanism replaces, by a comparison that F decides; and a *check* of the pair by a judge of family F .
- **Exogenous-family evidence.** The verdict is decided on the representation of a family N that separates part of what F does not separate: $\Sigma_F \setminus \Sigma_N$ has positive mass under the fault distribution in use.
- **Reality-grounded evidence.** The verdict is determined by ground truth for φ on a covered scope: acceptance by the downstream consumer, outcomes at run time, execution against the real environment, or a proof against a specification that realizes φ on that scope.

This is not a rung-by-rung version of Part III’s ladder, which sorts evidence by who ran the check. A second vendor can share the blind spot. A proof can establish a specification that misses the intent.

Definition 10 (Regression). A *regression* changes $\kappa_a(v, S)$ from CLOSED to OPEN or UNKNOWN when a reopening condition holds: an assumption of the mechanism is invalid, a counterexample is found, or the requirements, the environment, or the policy change. Every closed axis carries its reopening conditions. The *withdrawal* of a mechanism ends its authority. A withdrawal is a regression if an executor then serves the axis. If another accepted mechanism becomes authoritative, the state stays CLOSED; this is a *replacement*, not a regression. If nothing serves the axis, the axis is blocked.

When acceptance regresses, closure claims supported by it lose that support. The fallback needs a valid acceptance.

2.7 Intent projection: decision closure of decomposition

Definition 11 (Plan, local intent, projection). A *global intent* I_0 is the want at the top layer; it can be informal. A *plan* $\mathcal{P}$ is a dependency graph whose nodes carry a *role* from a fixed vocabulary and whose edges carry *licenses*: the local costs that the edge authorizes. A *local intent* I_n is what the worker at node n must want. It is *complete* if the worker can act correctly from I_n alone, and *blind* if it contains no more of the parent intent than the plan licenses. It is a *restriction* of its parent if its constraints and its authority do not exceed those of the parent. The *projection* is the map $(I_{\text{parent}}, \mathcal{P}, n) \mapsto I_n$. It is indexed by the plan.

Projection is recursive; the tree can grow as work splits. We assume a permissions representation with a terminating containment check. Containment of arbitrary policies need not be decidable. Objective refinement can still require judgment, even when authority containment is checked. A check is closed only where a mechanism is authoritative.

Definition 12 (Law, catalog, residue). A *law* is a rule that maps a pair (shape of the parent intent, plan role) to a pair (local intent, local acceptance). A *law catalog* is a versioned set of laws, read as a rewrite system on intent terms. When two laws match, the more specific law applies; then the law with the higher explicit priority; otherwise the step is escalated. A projection records the versions of the laws that produced it. The *residue* is the set of projection steps whose result the catalog does not determine: the steps that no law covers, and the steps on which matching laws stay in conflict.

We require compatible sibling acceptances, checked on the result, but not confluence. A terminating catalog mechanism closes the decision it determines. Residue steps are decision holes. Their acceptance combines decidable structural checks with judgment that each leaf serves its parent. Replacing a recurring residue step by a law is promotion. Regression can enlarge either the projection residue or the production residual.

Definition 13 (Two kinds of blindness; intent leakage). A worker is *informationally blind* if its work surface gives it no access to the global intent or to global data. A worker is *authority-blind* if it holds no license to act on the global intent: every local cost that it incurs cites a plan edge, and an action on an unlicensed rationale fails acceptance. *Intent leakage* occurs when a worker holds enough of the global objective, without the plan, to make a decision that is locally rational and contradicts the plan.

A work surface can restrict data and tools, but not training knowledge. Authority blindness proposes a contract check on unlicensed action. We have no worked example showing an additional fault it catches.

For Proposition 6, acceptance has a decidable *formal* part post^f , served by closed acceptance, and a *residual* part post^r , undecidable by any accepted mechanism and discharged by gated judgment with nonzero escape.

2.8 Context load and bounded holes

Definition 14 (Context load, window, load-dependent hazard). The *context load* $\ell \geq 0$ of a call is the amount of state that the call must use, in the unit of the window. The *window* K is the largest load that one call accepts. The *load-dependent hazard* $\lambda(\ell)$, for $0 \leq \ell \leq K$, is the fault hazard of the executor when its calls have the load ℓ , for a fixed measure of the load and a fixed distribution of the work.

Part I holds the hazard at the load in use constant.

Definition 15 (Bounded hole, bounded projection). A semantic hole is ℓ_h -*bounded* if every call through it has a load of at most ℓ_h . A projection is ℓ_h -*bounded* if every local intent that it produces gives, with its work surface, an ℓ_h -bounded hole.

Completeness and blindness do not imply bounded load: a plan can license a large necessary input. A complete bounded projection need not exist. State outside holes remains in plans, mechanisms, and artifacts. Projection itself consumes work; its faults enter f_π in Proposition 6.

External storage also exists in model-only systems. The distinction is authority on the axes. An agent is *fully open* when all three axes are open on every step: the model decides, produces, and judges completion. Open production under closed acceptance is not fully open.

3 Results

3.1 R-A: horizon under partial closure

Assumption 1 (Series assumptions on open segments). *On segments that an executor produces, the assumptions of Part I hold (checkpoint completeness, memoryless retries and segments, conditional gate independence), and the gate family has stealth mass $\lambda_{\text{st}} > 0$. The slip probability of an open segment is ε_o ; by Part I, $\varepsilon_o \geq p \lambda_{\text{st}}$.*

Assumption 2 (Independent segments, stationary mixing). *All segments have the length s . Each segment has a vector of random elements: its assignment to the open or the closed path, its input, the randomness of its gates, and its outcomes on the open path and on the fallback path. The vectors of different segments are mutually independent and have one law. In that law the segment is routed to a mechanism with probability c , and the input of a routed segment is a draw from $\mathcal{D}(\cdot \mid S)$. For regions, the segment belongs to region i with probability w_i . A run succeeds if no segment slips.*

Assumption 3 (Fixed mechanisms, no drift). *The deployed mechanisms, the predicate φ , and the distribution $\mathcal{D}$ do not change over the horizon.*

Assumption 4 (Escapes are faults; rejected outputs fall back). *A premature-closure fault that escapes makes its segment bad in the sense of Part I. An output of a mechanism that the retained stack rejects sends the segment to the open path. The retained stack is deterministic.*

Here c is the routing share of §2.4: a segment is first sent to a mechanism with probability c .

Proposition 1 (R-A: horizon under partial closure). *Under Assumptions 1–4, let $\varepsilon_c = p \lambda_{\text{pc}}$ be the probability that a routed segment has an escape (the mean over the routing scopes, weighted by volume, when several mechanisms serve the routed share), and let*

$$\lambda_{\text{mix}} = (1 - c) \lambda_{\text{st}} + c \lambda_{\text{pc}} > 0.$$

Then for every gate count,

$$\text{success}(T) \leq (1 - p \lambda_{\text{mix}})^{T/s}, \quad \text{hence} \quad H \leq \frac{s \ln 2}{-\ln(1 - p \lambda_{\text{mix}})} \leq \frac{s \ln 2}{p \lambda_{\text{mix}}} = \frac{H_{\text{raw}}}{\lambda_{\text{mix}}} (1 + O(\lambda s)).$$

For regions i with volume shares w_i , where region i is open with stealth mass $\lambda_i = \lambda_{\text{st}}^{(i)}$ or closed with escape $\lambda_i = \lambda_{\text{pc}}^{(i)}$, the same bound holds with $\lambda_{\text{mix}} = \sum_i w_i \lambda_i$. The form $H_{\text{raw}}/\lambda_{\text{mix}}$ holds to first order in λs only; for segments that are not short, use the first two forms.

Proof. The mechanisms are fixed. An open segment slips with probability at least $p \lambda_{\text{st}}$. A routed segment slips on its escape set, of probability $\varepsilon_c = p \lambda_{\text{pc}}$ under $\mathcal{D}(\cdot \mid S)$. Mixing gives $q \geq p((1 - c) \lambda_{\text{st}} + c \lambda_{\text{pc}}) = p \lambda_{\text{mix}}$. Each slip is a function of its segment's vector, so Assumption 2 gives independent, identically distributed slips. Hence $\text{success}(T) = (1 - q)^{T/s} \leq (1 - p \lambda_{\text{mix}})^{T/s}$. At half success,

$$H \leq \frac{s \ln 2}{-\ln(1 - p \lambda_{\text{mix}})} \leq \frac{s \ln 2}{p \lambda_{\text{mix}}},$$

using $-\ln(1 - x) \geq x$. Since $p = \lambda s(1 + O(\lambda s))$ and $H_{\text{raw}} = \ln 2/\lambda$, this gives the first-order form. For regions, average the lower slip bounds $p \lambda_i$ with weights w_i and repeat. $\square$

To first order, $c = 0$ recovers Part I; $\lambda_{\text{pc}} = 0$ raises the ceiling by $1/(1 - c)$; and $c = 1$ gives $H_{\text{raw}}/\lambda_{\text{pc}}$ when $\lambda_{\text{pc}} > 0$. Closing region i raises the ceiling exactly when $\lambda_{\text{pc}}^{(i)} < \lambda_{\text{st}}^{(i)}$.

The realized-slip criterion differs. Let b be the retained stack's rejection mass, including detected faults and false rejections. Let ε_{fb} be the open-path slip conditional on fallback. Rejection selects inputs, so it need not equal ε_o . The closed slip is $\varepsilon_c + b \varepsilon_{\text{fb}}$. Closure improves it exactly when this is below ε_o . If $\varepsilon_{\text{fb}} = \varepsilon_o$, the criterion is $\varepsilon_c < (1 - b) \varepsilon_o$; $\lambda_{\text{pc}} < (1 - b) \lambda_{\text{st}}$ is sufficient.

Independence is essential. Without fallback, associated escape indicators give a no-escape probability at least the product form, by Part III. Positive dependence of inputs alone is insufficient. With one repeated input, success is $1 - \varepsilon_c$ at every horizon and can exceed the bound. Random mixing and independent inputs also idealize a graph whose sites consume upstream outputs.

Remark 2 (The bound is conditional on the mechanism). Proposition 1 uses the realized λ_{pc} of the deployed mechanism. Suppose instead that only the expectation $\bar{\lambda}_{\text{pc}}$ over a build process is known. The bound $(1 - p((1 - c)\lambda_{\text{st}} + c\lambda_{\text{pc}}))^{T/s}$ is convex in λ_{pc} . By Jensen's inequality its expectation over mechanisms is at least its value at $\bar{\lambda}_{\text{pc}}$. The value at the mean is therefore not an upper bound on the expected success in general: the mean does not justify the plug-in bound. Weaker bounds that use the mean only exist, and the distribution of λ_{pc} gives sharper ones. This limits the use of Proposition 4, which is a statement in expectation, as an input to Proposition 1.

3.2 R-B: what promotion evidence can show

A mechanism authored by F decides through code, not through F . Part II therefore constrains same-family promotion evidence, not the mechanism's error in general.

Fix a site, production axis, and countable scope S with the discrete σ -algebra. Let $\mathcal{Z} = S \times Y$, $\mathcal{Z}_{\text{good}} = \{z : \varphi(z) = 1\}$, and $\mathcal{Z}_{\text{bad}} = \{z : \varphi(z) = 0\}$. The representation r_F records everything F computes about a pair across the admissible configurations of Part II. Write $z \sim_F z'$ iff $r_F(z) = r_F(z')$ and $\Sigma_F = \{z \in \mathcal{Z}_{\text{bad}} : \exists z' \in \mathcal{Z}_{\text{good}}, z \sim_F z'\}$. Let $\mathcal{M}$ be the mechanism class.

Definition 16 (Evidence). An *evidence procedure* is a map e from $\mathcal{Z}$ to a finite set of verdicts. An *evidence set* $\mathcal{E}$ is a finite set of evidence procedures. For a mechanism M and sample inputs $x_1, \dots, x_n$, the *evidence record* is the list of the inputs x_j and of the verdicts $e(x_j, M(x_j))$ for $e \in \mathcal{E}$. A procedure is *decided within F* if it is constant on the classes of $\sim_F$.

This evidence is pointwise. A review that reads the mechanism's text is outside the definition.

Definition 17 (Twin mechanisms and twin mass). Two mechanisms $M, M' \in \mathcal{M}$ are $\mathcal{E}$ -*twins* if $e(x, M(x)) = e(x, M'(x))$ for every $x \in S$ and every $e \in \mathcal{E}$. They are F -*twins* if $(x, M(x)) \sim_F (x, M'(x))$ for every $x \in S$. The *twin masses* of M are

$$\tau_{\mathcal{E}}(M) = \sup\{\pi(M') : M' \text{ is an } \mathcal{E}\text{-twin of } M\}, \quad \tau_F(M) = \sup\{\pi(M') : M' \text{ is an } F\text{-twin of } M\}.$$

Both masses are at least $\pi(M)$, since M is its own twin.

Assumption 5 (Evidence record). *The sample inputs are independent draws from $\mathcal{D}(\cdot \mid S)$. The promotion predicate, and every bound that is computed for a promotion, is a function of the evidence record and of auxiliary randomness. The joint law of the sample and the auxiliary randomness is the same for every mechanism in the class. It is not sufficient that the sample and the auxiliary randomness are each independent of the mechanism.*

Assumption 6 (Representation restriction). (i) On the evidence. *The map r_F ranges over all admissible configurations of a fixed parameter set, as in Part II; a change of configuration selects within this capacity and does not extend it.* (ii) On the mechanisms. *The class $\mathcal{M}$ is closed under finite patches: if $M \in \mathcal{M}$, and M' is a map that differs from M on finitely many inputs, then $M' \in \mathcal{M}$. A finite patch is a table in front of the procedure, so M' is a mechanism in the sense of Definition 3. We do not assume closure under arbitrary maps: a class of procedures is countable, and the maps can be uncountably many.*

Proposition 2 (Evidence does not separate twins). *Under Assumption 5, let $\mathcal{E}$ be an evidence set.*

- (a) **Indistinguishability.** *If M and M' are $\mathcal{E}$ -twins, they have the same evidence record on every sample. A promotion predicate therefore promotes M and M' with the same probability.*

(b) **Non-certifiability.** Let U be a bound that is valid over the class: $\Pr[\pi(M) \leq U] \geq 1 - \eta$ for every $M \in \mathcal{M}$. Then for every $M \in \mathcal{M}$ and every $t < \tau_{\mathcal{E}}(M)$, $\Pr[U \geq t] \geq 1 - \eta$ when the mechanism is M . The bound can be below t with a probability of at most η .

(c) **Monotonicity.** If $\mathcal{E} \subseteq \mathcal{E}'$, then $\tau_{\mathcal{E}'}(M) \leq \tau_{\mathcal{E}}(M)$.

Proof. (a) Twin verdicts agree on every sampled input. Assumption 5 gives the record and auxiliary randomness the same joint law, hence the same promotion probability. (b) For $t < \tau_{\mathcal{E}}(M)$ choose a twin M' with $\pi(M') > t$. By (a), U has the same law for both mechanisms. Validity for M' gives $\Pr[U \geq \pi(M')] \geq 1 - \eta$, hence $\Pr[U \geq t] \geq 1 - \eta$. (c) Adding procedures restricts the twin class over which the supremum is taken. $\square$

Corollary 1 (Same-family evidence). *Let every procedure in $\mathcal{E}$ be decided within F . Then every F -twin of M is an $\mathcal{E}$ -twin of M , so $\tau_{\mathcal{E}}(M) \geq \tau_F(M)$, and Proposition 2(b) holds for every $t < \tau_F(M)$: a valid bound that rests on the evidence is below t with a probability of at most η . This holds for every sample size and for every number of procedures that are decided within F . If in addition Assumption 6(ii) holds, then*

$$\tau_F(M) = \mathcal{D}(\{x \in S : \exists y' \text{ with } (x, y') \in \mathcal{Z}_{\text{bad}}, (x, y') \sim_F (x, M(x))\} \mid S),$$

as a supremum that need not be attained. When the retained acceptance is empty, $\varepsilon_c = \pi(M)$, and the same holds for λ_{pc} and $t < \tau_F(M)/p$.

Proof. Procedures constant on $\sim_F$ give equal verdicts on F -twins, so Proposition 2(b) applies. Let T be the displayed set. Every fault of an F -twin lies in T , giving $\pi(M') \leq \mathcal{D}(T \mid S)$. For $\epsilon > 0$, countability gives finite $T_\epsilon \subseteq T$ with $\mathcal{D}(T_\epsilon \mid S) \geq \mathcal{D}(T \mid S) - \epsilon$. Patch M on T_ϵ with the faulty y' in the display. The patch is an F -twin in $\mathcal{M}$ by Assumption 6(ii), with fault mass at least $\mathcal{D}(T_\epsilon \mid S)$. All sets are measurable in the discrete σ -algebra. Taking the supremum proves equality. With empty acceptance, divide $\varepsilon_c = \pi(M)$ by p . $\square$

Assumption 6(i) includes new prompts, deeper same-family judging, and same-family checks of verdicts, provided they judge pairs. The result concerns invariance, not achievability. Twin mass is an input mass under $\mathcal{D}$; λ_F is a fault mass under μ . The corollary does not relate them or show that twin mass is large.

Added evidence can only lower twin mass. Evidence decided within N still leaves simultaneous F - and N -twins indistinguishable; it need not separate all N classes. A grounded procedure returning φ on $S_g \subseteq \mathcal{Z}$, and no verdict elsewhere, prevents a twin from having a covered fault that M lacks. Strict improvement depends on which pairs the procedures separate, not vendor identity or stealth-set equality alone.

Proposition 3 (Exact replay transfers the fault mass of the reference). *Let $y_{\text{ref}} : S \rightarrow Y$ be a reference that is defined on all of S , with fault mass $\varepsilon_{\text{ref}} = \mathcal{D}(\{x : \varphi(x, y_{\text{ref}}(x)) = 0\} \mid S)$, and let $\delta = \mathcal{D}(\{x : M(x) \neq y_{\text{ref}}(x)\} \mid S)$. Then $|\pi(M) - \varepsilon_{\text{ref}}| \leq \delta$.*

Proof. The fault indicators agree wherever $M(x) = y_{\text{ref}}(x)$. Their difference is supported on a set of mass δ . $\square$

A recorded reference supports this statement only on its recorded scope. Grounded bounds likewise apply only on their covered scope; δ itself needs estimation.

For accepted-output replay, let independent retry rounds at input x produce a fault with probability p_x and conditional law μ_x . A stack G_0 accepts with positive probability; the first accepted

candidate is emitted. For $x \sim \mathcal{D}(\cdot \mid S)$, write $p = \mathbb{E}_x[p_x]$ and $p\lambda_F = \mathbb{E}_x[p_x\mu_x(\Sigma_F)]$. These are raw, pre-gate quantities; with one site per segment, p is the raw segment fault probability used elsewhere. A stack is *sound in the sense of Part II* if it has zero false-reject mass under a distribution on $\mathcal{Z}_{\text{good}}$ giving positive mass to every $\sim_F$ class meeting $\mathcal{Z}_{\text{good}}$. Zero false rejection under the executor's output distribution alone is insufficient.

Corollary 2 (A copy of accepted outputs inherits the floor). *Let G_0 be decided within F and sound in the sense of Part II, and let $y_{\text{ref}}(x)$ be the emitted output for x , drawn once per input. Let M disagree with y_{ref} on a mass δ , and let the retained stack G be empty, or decided within F and sound in the sense of Part II. Then*

$$\mathbb{E}[\varepsilon_c] \geq p\lambda_F - \mathbb{E}[\delta], \quad \text{equivalently} \quad \mathbb{E}[\lambda_{\text{pc}}] \geq \lambda_F - \mathbb{E}[\delta]/p,$$

where the expectation is over the reference.

Proof. Part II's sound inheritance theorem makes G_0 accept every pair in Σ_F . If $A_x \leq 1$ is the round acceptance probability, independent retries give an emitted stealth pair with probability $p_x\mu_x(\Sigma_F)/A_x \geq p_x\mu_x(\Sigma_F)$. Averaging gives expected reference stealth mass at least $p\lambda_F$. Where M agrees with that reference, the pair is faulty and G accepts it, by the same theorem or because G is empty. Disagreement removes at most mass δ . Take expectations and divide by p . $\square$

This is an expectation over references, not a floor for every deployed copy. We have not developed the false-rejection tolerance case. The statement does not cover generalization with large disagreement.

Definition 18 (Build process and produced faults). A *build process* $\mathbf{B}$ is a random procedure that returns a mechanism in $\mathcal{M}$. Its *produced fault mass* is $\pi_{\mathbf{B}} = \Pr[\varphi(x, M_0(x)) = 0]$ for $M_0 \sim \mathbf{B}$ and $x \sim \mathcal{D}(\cdot \mid S)$, with x independent of M_0 . For $\pi_{\mathbf{B}} > 0$ the *produced-fault distribution* $\mu_{\mathbf{B}}$ is the law of $(x, M_0(x))$ given $\varphi(x, M_0(x)) = 0$; it is a distribution on $\mathcal{Z}_{\text{bad}}$. The *produced stealth share* is $\lambda_F^{\mathbf{B}} = \mu_{\mathbf{B}}(\Sigma_F)$.

The set Σ_F describes what evidence cannot detect; $\mu_{\mathbf{B}}$ describes what the builder produces. Their overlap $\lambda_F^{\mathbf{B}}$ can be any value in $[0, 1]$.

Definition 19 (Undetected set). Let the procedures in $\mathcal{E}$ return a verdict in $\{\text{agree}, \text{disagree}\}$, and let G be the retained stack. The *combined procedure* accepts a pair if every procedure in $\mathcal{E}$ agrees and every gate in G accepts. The *undetected set* $\mathcal{U} \subseteq \mathcal{Z}_{\text{bad}}$ is the set of faulty pairs that the combined procedure accepts.

Assumption 7 (Repair leaves undetected faults in place). *Let $M_0 \sim \mathbf{B}$ be the mechanism that is submitted first. Every submitted mechanism is repaired until it is promoted, so every build ends with a mechanism M in service. For every input x with $(x, M_0(x)) \in \mathcal{U}$, $M(x) = M_0(x)$.*

Repair can violate this assumption by changing other outputs. Dropping builds with more undetected faults can also invalidate the conclusion for deployed builds.

Assumption 8 (False rejection). *The combined procedure has a false-reject mass of at most ρ_0 under a distribution ν on $\mathcal{Z}_{\text{good}}$ that gives positive mass to every class of $\sim_F$ that meets $\mathcal{Z}_{\text{good}}$ (so these classes are countably many, as in Part II). For $\rho_0 > 0$ the uniform likelihood-ratio bound of Part II holds with a finite constant κ , with $\mu_{\mathbf{B}}$ in the place of the fault distribution.*

Proposition 4 (Produced faults that nothing detects). *Under Assumption 7, with $\pi_B > 0$, in expectation over the builds,*

$$\mathbb{E}[\varepsilon_c] \geq \pi_B \mu_B(\mathcal{U}).$$

If every procedure in $\mathcal{E}$ and every gate in G is decided within F , and Assumption 8 holds, then $\mu_B(\mathcal{U}) \geq \lambda_F^B - \kappa\rho_0$, with $\kappa\rho_0 = 0$ in the sound case $\rho_0 = 0$. Hence

$$\mathbb{E}[\varepsilon_c] \geq \pi_B(\lambda_F^B - \kappa\rho_0), \quad \text{equivalently} \quad \mathbb{E}[\lambda_{pc}] \geq \frac{\pi_B}{p}(\lambda_F^B - \kappa\rho_0).$$

Proof. If $(x, M_0(x)) \in \mathcal{U}$, repair preserves the faulty output and G accepts it. Thus $\varepsilon_c \geq \mathcal{D}(\{x : (x, M_0(x)) \in \mathcal{U}\} \mid S)$. Its expectation is $\Pr[(x, M_0(x)) \in \mathcal{U}] = \pi_B \mu_B(\mathcal{U})$. For the second part, the combined procedure is constant on $\sim_F$ classes. Part II's inheritance theorem puts Σ_F in its blind set $\mathcal{U}$ except for μ_B -mass at most $\kappa\rho_0$. Substitute and divide by p . $\square$

This lower bound need not be equality: repairs can add faults, and detectable faults can miss the evidence sample.

Hypothesis 1 (Distillation by a builder that generalizes). *A builder of family F that generalizes from the accepted outputs of an executor of family F , and does not copy them, produces at least as much stealth fault mass as the raw executor: $\pi_B \lambda_F^B \geq p \lambda_F$, with p and λ_F the raw quantities of the executor, before its gates.*

Under this hypothesis and the repair and false-rejection assumptions, and with every evidence procedure and retained gate decided within F , Proposition 4 gives $\mathbb{E}[\lambda_{pc}] \geq \lambda_F - (\pi_B/p)\kappa\rho_0$. We have no evidence for or against the hypothesis. Reports about shared checker blind spots do not establish what a mechanism builder produces. Without the hypothesis, the proposition still holds with the builder's quantities.

A machine-executed test authored by F has execution as its deciding substrate. It is grounded only where its assertions realize φ . The same-family conclusions require an additional *authored-coverage assumption*: the check never separates a fault in Σ_F from its acceptable twin. That empirical assumption can fail. Proposition 2 and the first inequality of Proposition 4 still apply to any such pointwise check.

Fixed prior knowledge does not evade class-wide validity. It must restrict the class, separate twins, or change the requirement, for example to validity averaged over builds. Restricted mechanism languages can have smaller twin masses. Until τ_F is estimated, a ban on promotion by one family's evidence is a heuristic: the model motivates it where the required fault mass is below τ_F .

3.3 R-C: mechanization break-even

Assumption 9 (Cost model). *A region receives calls at a constant rate r and stays closed for a lifetime τ before a regression. Per served call, the open path costs $k_o = a(c_x + g)$, with c_x the cost of an executor call, g the cost of the gate stack, and a the expected number of attempts. The closed path costs $k_c = c_m + g' + b k_o$, with c_m the cost of a mechanism call, g' the cost of the retained acceptance, and b the fallback mass of §3.1. A segment that falls back slips on the open path with probability ε_{fb} (§3.1). An escape costs L . Closure has a one-time build cost B , a one-time validation cost V , and an upkeep u per unit time. Costs of different regions add.*

Assumption 10 (Sampled validation). *A promotion needs a certificate. The sample is n independent draws from $\mathcal{D}(\cdot \mid S)$, independent of the mechanism. The evidence is exact on the sample: it reports a fault on an input if and only if the mechanism has a fault on that input. The rule promotes*

if and only if the sample shows no fault. The rule promotes a mechanism with a fault mass above θ with probability at most η . The class $\mathcal{M}$ contains mechanisms with fault masses arbitrarily close to θ from above. Each evidence item that is not same-family evidence costs c_e .

Proposition 5 (R-C: break-even with the validation cost). *Under Assumption 9, let the saving per served call be*

$$\Delta = (k_o + \varepsilon_o L) - (k_c + (\varepsilon_c + b\varepsilon_{fb})L).$$

- (a) **Break-even.** *The closed path has the lower total cost over the lifetime if and only if $(r\Delta - u)\tau > B + V$. The break-even volume is*

$$N^* = \frac{B + V + u\tau}{\Delta} \quad (\Delta > 0),$$

which is $(B + V)/\Delta$ without upkeep.

- (b) **Sample size.** *Under Assumption 10, $n \geq \ln(1/\eta)/\ln(1/(1 - \theta))$. If the items are not same-family evidence,*

$$V \geq c_e \frac{\ln(1/\eta)}{\ln(1/(1 - \theta))} \geq c_e \frac{(1 - \theta) \ln(1/\eta)}{\theta}.$$

- (c) **Frontier.** *For regions i with parameters $(r_i, \tau_i, \Delta_i, B_i, V_i, u_i)$, a closed set with the lowest total cost is $\{i : (r_i \Delta_i - u_i)\tau_i > B_i + V_i\}$; a region with equality can be open or closed. The closed share is below 1 if some region with a positive volume share has $(r_i \Delta_i - u_i)\tau_i < B_i + V_i$.*

Proof. (a) Subtract the closed cost $B + V + u\tau + r\tau(k_c + (\varepsilon_c + b\varepsilon_{fb})L)$ from the open cost $r\tau(k_o + \varepsilon_o L)$. The difference is $(r\Delta - u)\tau - (B + V)$; solving at zero gives N^* . (b) A mechanism of fault mass $\theta' > \theta$ is promoted with probability $(1 - \theta')^n$. Validity requires this to be at most η . Taking $\theta' \downarrow \theta$ within the class gives $(1 - \theta)^n \leq \eta$ and the sample bound. Multiply by c_e and use $\ln(1/(1 - \theta)) \leq \theta/(1 - \theta)$. (c) Additive costs are minimized by selecting each region's cheaper path, with either choice at equality. $\square$

If $\theta < \tau_F(M)$, a pointwise same-family rule promotes M as often as a twin above θ . A valid certificate therefore promotes M with probability at most η . Promotion above that probability needs other evidence; part (b) prices it when sampled. A proof need not have this sample-cost scaling. The suggestion that a model builder lowers B is unmeasured.

3.4 R-D: composition

A parent with acceptance $(\text{post}_p^f, \text{post}_p^r)$ has children $1, \dots, k$ under plan $\mathcal{P}$. It is accepted when the projection and all children are accepted and the trace satisfies the decidable sequencing predicate $\text{Seq}(\mathcal{P})$. Let I_i indicate that child i is accepted with post_i^f false, with $f_i = \Pr[I_i = 1]$. Let I_π indicate an escaped projection fault, with $f_\pi = \Pr[I_\pi = 1]$.

Assumption 11 (Formal coverage). *The checker of each formal part is correct: a child that is accepted satisfies post_i^f . The implication*

$$\left(\bigwedge_{i=1}^k \text{post}_i^f\right) \wedge \text{Seq}(\mathcal{P}) \implies \text{post}_p^f$$

holds for every projection that is accepted.

Assumption 12 (Residual completeness). *If the projection has no escaped fault and post_i^f holds for every child, then post_p^r holds.*

Assumption 13 (Monotone provenance). *The escape indicators in the statement are coordinate-wise non-decreasing functions of mutually independent primitive fault variables: $I_\pi, I_1, \dots, I_k$ for one parent, and the indicators of all projections and all leaves for a tree.*

Part III supplies the one-layer bound, not this assumption for a tree with internal gates. For one parent the assumption includes internal children.

Remark 3 (The formal part). Under Assumption 11, an accepted parent satisfies post_p^f : its children are accepted, so every post_i^f holds; the trace satisfies $\text{Seq}(\mathcal{P})$; and the implication gives post_p^f . This restates the assumption and is not a result.

Proposition 6 (R-D: composition of the residual part).

(a) **One parent.** Under Assumptions 12 and 13,

$$\Pr[\text{the parent is accepted and } \text{post}_p^f \text{ is false}] \leq 1 - (1 - f_\pi) \prod_{i=1}^k (1 - f_i).$$

(b) **Depth.** Let an intent tree be fixed and finite. Let every internal node satisfy Assumption 12, let Assumption 13 hold for the tree, and let the children of an accepted node be accepted. Then the probability that the root is accepted and its residual part is false is at most $1 - \prod_j (1 - f_j)$, where j ranges over all leaves and all projections.

Proof. (a) Completeness makes an accepted false parent imply an escaped projection or a false residual part at some child. That child is accepted, so its indicator is 1. The parent event lies in $\{I_\pi = 1\} \cup \bigcup_i \{I_i = 1\}$. By monotone provenance and Part III, $\Pr[I_\pi = 0, I_1 = 0, \dots, I_k = 0] \geq (1 - f_\pi) \prod_i (1 - f_i)$. Take complements. (b) Acceptance descends from the root to every child. At each internal node with false residual part, completeness gives an escaped projection or a child with false residual part. Induction on finite depth puts the root event inside the union of all projection and leaf escapes. Apply Part III to that union. $\square$

Completeness carries most of the weight: conflicts among acceptable children count as projection faults. The result locates them in f_π without making it small. It compares dependence at fixed marginals, not the benefit of shared provenance, and gives no lower bound. A projection fault can change which children exist, so monotonicity is not assured.

3.5 R-E: scale

Corollary 3 (Reach and residual). Under Assumptions 1–4, let each segment slip with probability $q(c) = (1 - c)\varepsilon_o + c(\varepsilon_c + b\varepsilon_{fb})$, with $0 < q(c) < 1$. The half-success horizon is

$$H(c) = \frac{s \ln 2}{-\ln(1 - q(c))}.$$

Let ε_o , ε_c , b , and ε_{fb} stay fixed when c changes. Then $H(c)$ grows with the routing share c if and only if $\varepsilon_c + b\varepsilon_{fb} < \varepsilon_o$. If the routed segments are served without a fault and without a rejection ($\varepsilon_c = 0$, $b = 0$, so $c = c_{\text{prod}}$) and $c < 1$, then

$$\frac{H(c)}{H(0)} \geq \frac{1}{1 - c},$$

and the ratio is asymptotically equivalent to $1/(1 - c)$ as $\varepsilon_o \rightarrow 0$.

Proof. Independence gives $\text{success}(T) = (1 - q(c))^{T/s}$; solve at one half. The affine $q(c)$ has slope $\varepsilon_c + b\varepsilon_{fb} - \varepsilon_o$, and H decreases in q . For $\varepsilon_c = b = 0$, let $g(x) = -\ln(1 - x)$. Convexity and $g(0) = 0$ give $g((1 - c)\varepsilon_o) \leq (1 - c)g(\varepsilon_o)$, hence the ratio bound. Finally $g(x) \sim x$ as $x \rightarrow 0$ gives the asymptotic equivalence. $\square$

A fault-free mechanism alone is insufficient for the ideal gain. If $b\varepsilon_{fb} > 0$, fallback keeps the horizon finite as $c \rightarrow 1$.

Hypothesis 2 (Load monotonicity). *The hazard $\lambda(\ell)$ of an executor does not decrease in the load ℓ , for $0 \leq \ell \leq K$.*

Measurements on models of 2023–2025 report retrieval degradation within longer contexts, depending on model, task, and information position [13, 14]. They do not measure long-work fault hazard or establish its monotonicity. We know of no measurement of hazard as a function of load.

Proposition 7 (R-E: bounded load). *Let faults arrive as a Poisson process whose rate at elapsed work t is the hazard at the load in use. The clock t counts the work of the executor; for a work program it counts the open work and not the closed work. The ungated horizon is $H = \inf\{T \geq 0 : \int_0^T \lambda(\ell(t)) dt \geq \ln 2\}$, with $\inf \emptyset = \infty$: the elapsed work at which the probability of no fault falls to one half, and ∞ if it never does. Let $0 < \lambda(\ell_h) \leq \lambda(\ell_0) \leq \lambda(K) < \infty$. Under Hypothesis 2:*

- (a) **One agent that carries the state.** *Let the load $\ell(t)$ of the agent not decrease in t , with $\ell(0) = \ell_0$ and $\ell(t) \leq K$. Its ungated horizon H_{mono} satisfies*

$$\frac{\ln 2}{\lambda(K)} \leq H_{\text{mono}} \leq \frac{\ln 2}{\lambda(\ell_0)}.$$

- (b) **A work program with bounded holes.** *Let every hole be ℓ_h -bounded with $\ell_h \leq \ell_0$, and let λ_h be the supremum of the hazard over its open work. Then $\lambda_h \leq \lambda(\ell_h)$, the hazard has a bound that does not depend on the elapsed work, and the ungated horizon of the open work is at least*

$$\frac{\ln 2}{\lambda(\ell_h)} \geq \frac{\lambda(\ell_0)}{\lambda(\ell_h)} H_{\text{mono}}.$$

- (c) **The gain is bounded for holes at one load.** *If every call of every hole has the load ℓ_h exactly, the ungated horizon of the open work is $\ln 2 / \lambda(\ell_h)$, and its ratio to H_{mono} is at most $\lambda(K) / \lambda(\ell_h)$. For holes that work below ℓ_h this upper bound does not hold.*

Proof. (a) No-fault probability is $\exp(-\int_0^T \lambda(\ell(t)) dt)$. Monotonicity and $\ell_0 \leq \ell(t) \leq K$ bound the integral between $\lambda(\ell_0)T$ and $\lambda(K)T$. Solving at $\ln 2$ gives both bounds. (b) Loads at most ℓ_h have hazard at most $\lambda(\ell_h)$. No-fault probability is at least $\exp(-\lambda(\ell_h)T)$, giving the first bound. The upper bound in (a) gives the comparison with H_{mono} . (c) Exact load ℓ_h gives constant hazard $\lambda(\ell_h)$ and horizon $\ln 2 / \lambda(\ell_h)$. Divide by $H_{\text{mono}} \geq \ln 2 / \lambda(K)$. $\square$

The closure gain compares gated horizons. The load gain compares ungated horizons. We do not multiply them: gates need not preserve hazard ratios. Bounded load gives neither constant hazard nor a clean context. Upstream faults, reset, and memorylessness remain separate issues. The comparisons fix the executor; load gain can be 1 when hazard is load-independent. They give no hazard for work requiring state beyond the window.

3.6 Kept open

We claim no result on these problems.

1. **Regression under drift.** A law for growth of λ_{pc} and a watch that bounds it.
2. **An estimator for λ_{pc} .** Fixed fault sets change what repeated observation identifies.
3. **Authored checks.** When checks authored by a family and executed by a machine satisfy the authored-coverage assumption.
4. **Decision and acceptance.** Their premature closure and its interaction with production.
5. **Intent leakage.** A benchmark with known leak points and opposing local and global intents.
6. **Authority blindness.** A worked example showing a fault it catches.
7. **Measurement.** Estimates of λ_{pc} , τ_F , λ_F^B , costs, and $\lambda(\ell)$; tests of the assumptions and Hypotheses 1 and 2.
8. **Closable share.** How much real work can close with small escape.

4 Simulation

We illustrate Propositions 1 and 4. **Every parameter is chosen, not measured.** The builders are stipulated, not evidence for or against Hypothesis 1. The standard-library Python script uses seed 20260930. All 38 sampling checks and 30 structural checks pass. Sampling checks compare Monte-Carlo values with exact values or bounds at stated tolerances. Structural checks hold by construction and guard code; they do not test propositions.

The executor has $\lambda = 0.02$, $s = 1$, $p \approx 0.0198$, and $H_{\text{raw}} \approx 34.7$. Family F has $\lambda_F = 0.15$. Each gate independently catches a visible fault with probability $q = 0.6$; drawn verdicts stay fixed. The open path uses 8 gates of F . Its escape per raw fault is $\lambda_8 = \lambda_F + (1 - \lambda_F)(1 - q)^8 \approx 0.1506$, which includes visible faults that all eight gates miss; it is not the shared stealth mass λ_{st} . As a stipulated finite-stack benchmark we take $\varepsilon_o = p\lambda_8$ and use λ_8 in the place of λ_{st} in the formulas of §3.1.

R-A. Table 2 uses independent segments. Closed escape has probability $p\lambda_{pc}$; rejection of mass b gives fallback slip ε_{fb} . Without fallback this is the ceiling’s equality case. Rows 1–3: to first order, fault-free, rejection-free closure of a share c raises the ceiling by $1/(1 - c)$ (the logarithmic ratios are 2.0015 and 10.013). Rows 4–6: at $\lambda_{pc} = \lambda_8$ the ceiling does not move, and at $\lambda_{pc} = 2\lambda_8$ it falls. Rows 7–8 compare fallback of mass $b = 0.2$ at the usual slip and at four times the usual slip. Fallback lowers realized success, especially when rejection selects harder inputs. Every row is checked against exact success with $q = (1 - c)\varepsilon_o + c(\varepsilon_c + b\varepsilon_{\text{fb}})$.

Promotion evidence. The scope has 2000 uniformly weighted inputs. Each build draws faults with probability π_B and stealth membership with probability λ_F^B . The combined procedures are an F -tower; depth-8 towers of F and N , with $\lambda_N = 0.15$ and overlap $\rho \in \{0, 0.5\}$; or a depth-8 F -tower with grounding on share g . Here $\rho = 0$ is independence and $\rho = 1$ maximal overlap. All procedures are sound, so $\kappa\rho_0 = 0$. The builder patches only rejected faults in a sample of 200 inputs. The combined procedure is retained. Thus Assumption 7 holds and, by construction, $\mathbb{E}[\varepsilon_c] = \pi_B\mu_B(\mathcal{U})$. The table is not a test of that accounting identity. With empty retained acceptance, every unrepaired fault escapes; escape is about 0.018 in each arm.

c	λ_{pc}/λ_8	b	$\varepsilon_{fb}/\varepsilon_o$	λ_{mix}	ceiling H	realized H	success(300): bound / realized
0	–	0	–	0.151	232	230	0.408 / 0.403
0.5	0	0	–	0.075	465	467	0.639 / 0.638
0.9	0	0	–	0.015	2325	2345	0.914 / 0.916
0.5	0.5	0	–	0.113	310	307	0.511 / 0.507
0.5	1	0	–	0.151	232	230	0.408 / 0.407
0.5	2	0	–	0.226	155	157	0.261 / 0.262
0.5	0.5	0.2	1	0.113	310	277	0.511 / 0.471
0.5	0.5	0.2	4	0.113	310	204	0.511 / 0.359

Table 2: Proposition 1 in the benchmark $\varepsilon_o = p\lambda_8$. The ceiling is the first form $s \ln 2 / (-\ln(1 - p\lambda_{mix}))$; the realized half-success horizon is the sample median of the first slip, over 40 000 runs. Realized values above the ceiling are sampling error: one standard error of the median is about 0.7% at this run count. The script checks each value against the bound with a tolerance of 3%.

evidence arm	builder B1: $\pi_B = p, \lambda_F^B = 0.15$		builder B2: $\pi_B = p, \lambda_F^B = 0.03$	
	$\mu_B(\mathcal{U})$	$\mathbb{E}[\lambda_{pc}]$	$\mu_B(\mathcal{U})$	$\mathbb{E}[\lambda_{pc}]$
F -tower, $h = 1$	0.490	0.488	0.418	0.414
F -tower, $h = 3$	0.204	0.204	0.092	0.092
F -tower, $h = 8$	0.151	0.152	0.031	0.030
+ exogenous $N, \rho = 0$	0.023	0.022	0.005	0.004
+ exogenous $N, \rho = 0.5$	0.086	0.086	0.017	0.016
+ grounded, $g = 0.5$	0.075	0.073	0.015	0.016
+ grounded, $g = 0.9$	0.015	0.015	0.003	0.003

Table 3: The share $\mu_B(\mathcal{U})$ of produced faults that the combined procedure accepts (closed form), and the mean premature-closure escape $\lambda_{pc} = \varepsilon_c/p$ over 1500 builds, with the combined procedure retained. Since $\pi_B = p$, $\mathbb{E}[\lambda_{pc}] = \mu_B(\mathcal{U})$ up to sampling error. The script rounds each entry from the unrounded value.

Deeper same-family towers approach the stipulated produced stealth share. Independent exogenous evidence and grounding reduce escape; overlap reduces the exogenous gain. B1 stipulates $\pi_B \lambda_F^B = p\lambda_F$, the hypothesis’s equality case. B2 stipulates a smaller produced stealth share. Neither estimates it.

For orientation, at $c = 0.5$ the logarithmic ceiling evaluated at mean λ_{pc} has ratio 1.00 to the open ceiling for B1 with depth-8 same-family evidence. The ratios are 1.75 and 1.27 with independent and overlapping exogenous evidence, and 1.35 and 1.82 with $g = 0.5$ and 0.9. These are not bounds on expected success. At $n = 233$, the mean conditional bound exceeds its value at mean λ_{pc} in every arm; the largest difference is 0.008. This illustrates Jensen’s inequality for the conditional bounds. This comparison does not simulate realized success with fallback.

5 The mechanization plan

This is a design. It has not run in this form. It gives no thresholds or measured rates.

5.1 States

Stages are recorded per axis and scope beside closure states. Candidate and shadow preserve the prior UNKNOWN or OPEN state. After withdrawal, an executor gives regression, an accepted

mechanism gives replacement, and no fallback gives blocking.

stage	meaning and authority
unclassified	Executor serves unexamined work; review is dated.
open	Executor serves work with a reviewed reason and review date.
candidate	Recurrence, review, a changed reason, or an available mechanism warrants examination; executor stays authoritative.
shadow	Mechanism runs under ratified criteria; outputs are recorded, not used.
closed	Promoted mechanism is authoritative.
reopened	Mechanism is withdrawn; accepted fallback serves, or the axis blocks.

Table 4: Plan stages. Executor-free service requires closure of every axis previously served by an executor.

5.2 Transitions and actors

Mechanical actors read events or fixed results; semantic actors are executors. An authority issues criteria or decides promotion.

transition	actor and action
new site $\rightarrow$ unclassified	Router creates a record and work item.
unclassified, open $\rightarrow$ candidate	Detector uses computed signatures, recurrence, review dates, or a new mechanism.
unclassified, candidate $\rightarrow$ open	Classifier records a reviewed reason and review date.
candidate $\rightarrow$ shadow	Builder proposes; a separate authority ratifies criteria before evaluation.
shadow $\rightarrow$ closed	Authority promotes on typed evidence.
shadow $\rightarrow$ candidate	Fixed comparison exceeds a disagreement bar; counterexamples go to classification.
closed $\rightarrow$ reopened	Watcher detects a reopening condition.
reopened $\rightarrow$ closed, open	New promotion, accepted replacement, or reviewed open judgment.

Table 5: Transitions. The builder and authority are separate roles.

5.3 Rules

1. Recurrence warrants examination, not promotion. Promotion requires ratified acceptance, explained shadow disagreements, recorded assumptions, and evidence beyond the executor’s family. This last requirement is policy. Corollary 1 motivates it where the required fault mass is below twin mass; it does not prove it necessary for every promotion.
2. Close acceptance before production on the scope. The model permits the reverse; this plan does not.
3. Shadow outputs are never authoritative.
4. The builder neither issues acceptance criteria nor decides promotion.
5. Refuse promotion without reopening conditions evaluable by a program on named events: revised assumptions or sources, failed acceptance, or reopened dependencies.

6. Promotion authority grows with blast radius. A low-radius class can have a shared bar.
7. Withdrawal reopens dependent axes and uses only an accepted fallback. An executor fallback has valid acceptance, including when the executor serves as a judge; its closure state is OPEN or UNKNOWN according to review. An accepted replacement remains CLOSED. Otherwise the axis blocks and raises work. The withdrawn mechanism may run without authority beside an executor until repromotion. For reopened acceptance, mark outputs accepted since the condition held for new acceptance.
8. Mechanical actors do not judge content.

5.4 Measures

The *mechanization rate* is the share of decisions made by programs over a window. The *unclassified surface* is the share on unclassified axes. Promotion and reopening rates count their events per period. The regression rate counts only reopenings followed by executor service. These decision counts differ from segment-weighted *c*. The program share of design nodes is only a structural proxy, since node frequencies differ.

5.5 What the plan does not say

Promotion and demotion lifecycles already exist [9]. This design adds the model’s axes, scopes, evidence classes, and authority separation. It does not specify mechanism discovery, candidate priority, worker context selection, or thresholds. Its costs and rates are unmeasured.

6 Instance

This instance reports counts and shapes, not modeled quantities or a test of a proposition. Counts were derived by script from system records on 2026-09-29 and can be re-derived from them; the records are private. One person operates language-model workers under written rules. Method and format repositories hold routing designs and typed records. Repository checks gate changes.

Routing designs. Five designs have 54 nodes: 20 **program**, 29 **agent**, and 5 **human**. The 34 executor sites cite no known constructor (14), no known decider (14), authority (5), or missing information (1). There are 29 recorded reviewed justifications and 5 unreviewed ones; the review record is an unsigned draft. We did not check whether each review establishes the axis-and-scope condition for OPEN. Labels are provisional: at most 29 nodes are OPEN and at least 5 are UNKNOWN. The program share $20/54 \approx 0.37$ is a structural proxy, not a mechanization rate.

Formats and mechanisms. There are 34 format and protocol definitions: 20 candidates, 11 deferred, and 3 proposals; none is ratified. An unadopted investigation classifies 99 fields in 17 formats: programs lead 63 decisions, models or people 35, and nothing 1.

Three implemented mechanisms have stated assumptions and residuals and run under repository checks. A spend-reservation meter replaces a manual spend clearance. Import reachability replaces the mechanical part of impact analysis; coupling beyond imports remains open. A check links each acceptance criterion to tests that changed from failing to passing; test adequacy and fidelity to the specification remain residual. None is adopted. No shadow run or promotion is recorded, so each is at most a candidate.

Reviews. From 2026-08-02 to 2026-09-26, 758 records cover 544 reviews. Records identify author and reviewer families and the issues that only one reviewer raised. In 25 reviews, both the author’s family and another family reviewed the same artifact. Same-family reviewers raised 52 such issues; other families raised 55. Other families raised at least one in 15 of these 25 reviews.

These records do not test a proposition. A raised issue is not a fault; ground truth is absent; the authoring family decided acceptance. Pairings were not randomized. Only one review had two reviewers from the author’s family, so there is no control separating family effects from a second-reviewer effect.

Unmeasured quantities. Of 47 repositories, 24 run check suites. Three of these suites invoke about 54, 78, and 28 checks. A check supports closed acceptance only where its verdict is deterministic and authoritative. The designed mechanization-rate measure is unimplemented and has no series. No λ_{pc} , τ_F , λ_F^B , or closable-share estimate is available.

7 Discussion

This section adds no result. Read together, the results make one argument, each step under its own conditions. Fault-free, rejection-free closure of a share c raises the modeled ceiling by $1/(1 - c)$ to first order (Proposition 1, Corollary 3); this is a lever outside the setting of Part I, in which the executor serves every segment, and it is distinct from diversity and grounding. Whether a closure is correct is a question for the promotion evidence, and pointwise evidence decided within one family cannot certify a fault mass below the twin mass, except with the allowed error probability (Corollary 1). A promotion that needs such a certificate, with probability above η , therefore needs evidence from outside the family, and that evidence has a cost (Proposition 5). In the model, executors are implementations of typed holes whose contracts and acceptance the program owns, not the architecture of the system.

The results connect closure, evidence, and cost under separate conditions. They do not rank the four closure combinations. The plan favors open production under closed acceptance, but a fixed checker can still miss intent.

The model names unmeasured failure modes: premature production closure, false acceptance, hidden assumptions, closure retained after drift, and growth of the residual. Reopening conditions address only recorded assumptions. The unknown share tracks unexamined work, not all residual growth: regression into OPEN enlarges the residual without increasing the unknown share.

Closure is relative to a bar. Under Assumption 10 the bar is the pair (θ, η) : the tolerated fault mass and the allowed error probability. Each domain sets its own bar: a fault mass that a game tolerates is unacceptable in avionics. By Proposition 5, a stricter bar needs more evidence, $n \geq \ln(1/\eta)/\ln(1/(1 - \theta))$. When evidence is priced per item as in Proposition 5(b), a stricter bar also raises the lower bound on V and, with the other cost parameters fixed and $\Delta > 0$, the lower bound on the break-even volume N^* . A closure certified under a bar meets every bar that is looser in both components, θ and η , if the mechanism, the intent predicate, the scope, the operating distribution, and the validation assumptions are the same; the converse need not hold. One body of formalized work can therefore serve several systems that share these conditions. Each system admits only the closures whose evidence meets its own bar. With the candidates and the evidence fixed, a stricter bar admits a subset of the closures, so a stricter system keeps at least as much of the work open. We call each such system a *variant*. This is a reading of the results, not a further result; the conditions above apply to it in full.

The author’s view, which the results do not test, returns to the analogy of the introduction. A

model is to a mechanized system roughly what mitochondria are to a complex cell: an ingredient the larger organism depends on, but not the organism itself. In the model, the executor is needed wherever semantics stay open. The results above state conditions under which closure raises the modeled ceiling and promotion evidence can support it.

8 Limitations

No modeled quantity is measured. The simulation’s parameters are chosen. The instance supplies counts only, and the plan has not run in the stated form.

The horizon ceiling assumes independent segments, fixed mechanisms, and no drift. Dependence can make realized success exceed the bound, so the ceiling need not hold. Evidence results use Part II’s fixed representation relation and the evidence-record assumption. Produced-fault bounds additionally require repair to preserve undetected faults. They concern expectations over builds; mean escape cannot be substituted into the conditional success bound. Pointwise evidence excludes text review and does not automatically include machine-executed checks in the author’s family.

Composition assumes residual completeness and monotone provenance for the tree. Neither is established for real projections. The cost model assumes additive regional costs and given lifetimes. Its sample bound requires exact, independent, zero-failure validation. It omits grounding latency and economies of scope; proofs need not incur the sample scaling.

Scale comparisons omit projection, interface, and acceptance overhead. They neither establish a complete bounded projection nor estimate the closable share. Their baselines differ. Both empirical hypotheses remain untested. Decision and acceptance closure are defined but not quantitatively analyzed. Adaptive adversarial interaction and strategic manipulation of evidence are outside the analysis. A fixed faulty mechanism, including a deliberate one, is covered where the assumptions hold.

9 Related work

LATM makes reusable tools for lighter models [15]. Compiled AI generates, validates, and runs code without a model [10]. TraceCompiler compiles agent traces into mostly deterministic workflows and declines to compile irreversible effects that the traces leave under-determined [16]. Progressive Crystallization reports production promotion and demotion lifecycles [9]. Its criteria include successful-run counts, action agreement, trace-derived tests, classification consistency, a model-free regression suite, and human logic review. Their classification here requires knowing deciding substrates and what assertions establish about φ . Successful traces count as grounded only where success verification is stated. Aggregate agreement is outside our evidence definition unless made pointwise; logic review is outside it. We claim nothing about that system’s faults. DSPy optimizes model-call pipelines, leaving production open [17].

Programs with holes come from sketching [4] and typed editors [5]; residual programs from partial evaluation [6]; compiled reasoning from knowledge compilation [7]; and untrusted production under checking from proof-carrying code [8]. We add closure axes, scopes, states, and their link to verification evidence.

Part II places shared blind spots in the multiversion-failure literature. A recent test-generation study reports 14% fault detection after exposure to faulty code, against 25% for independently generated tests [11]. This is lower, not zero, detection in that setting and does not measure our plan. Long-context retrieval studies motivate, but do not establish, the load hypothesis [13, 14].

10 Conclusion

In this model, routing a share of production to mechanisms raises the ceiling where $\lambda_{pc} < \lambda_{st}$. With path laws fixed, it improves realized horizon where $\varepsilon_c + b\varepsilon_{fb} < \varepsilon_o$. Promotion evidence must support that distinction, and pointwise same-family evidence has the twin limits proved here. The next steps are an escape estimator, a distillation-hypothesis test, and measurement of the closable share.

Code and data availability

The standard-library Python script `simulate_psm.py` uses seed 20260930 and reproduces Tables 2 and 3 and the simulation numbers. It is at <https://github.com/ivmat/gated-computation-sim>. Instance records are private. Companion papers are on Zenodo: Part I, [10.5281/zenodo.20820968](https://doi.org/10.5281/zenodo.20820968); Part II, [10.5281/zenodo.20837102](https://doi.org/10.5281/zenodo.20837102); Part III, [10.5281/zenodo.21456783](https://doi.org/10.5281/zenodo.21456783).

References

- [1] I. Matijašević. A Fault-Tolerance Threshold for Gated Agentic Computation: Reliable Long-Horizon Work from Unreliable Executors. 2026. Archived on Zenodo, DOI: [10.5281/zenodo.20820968](https://doi.org/10.5281/zenodo.20820968).
- [2] I. Matijašević. Generated Gates Inherit Their Generator’s Blind Spots: A Diversity Floor for Self-Generated Verification. 2026. Archived on Zenodo, DOI: [10.5281/zenodo.20837102](https://doi.org/10.5281/zenodo.20837102).
- [3] I. Matijašević. Escape, Cost, and Correlation at the Verification Floor of Gated Agentic Computation. 2026. Archived on Zenodo, DOI: [10.5281/zenodo.21456783](https://doi.org/10.5281/zenodo.21456783).
- [4] A. Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis, University of California, Berkeley, 2008.
- [5] C. Omar, I. Voysey, M. Hilton, J. Aldrich, and M. A. Hammer. Hazelnut: a bidirectionally typed structure editor calculus. In *POPL*, 2017.
- [6] N. D. Jones, C. K. Gomard, and P. Sestoft. *Partial Evaluation and Automatic Program Generation*. Prentice Hall, 1993.
- [7] A. Darwiche and P. Marquis. A knowledge compilation map. *J. Artificial Intelligence Research*, 17:229–264, 2002.
- [8] G. C. Necula. Proof-carrying code. In *POPL*, 1997.
- [9] A. Malik. Progressive Crystallization: Turning Agent Exploration into Deterministic, Lower-Cost Workflows in Production. *arXiv:2607.07052*, 2026.
- [10] G. Trooskens, A. Karlsberg, A. Sharma, L. De Brouwer, M. Van Puyvelde, M. Young, J. Thickstun, G. Alterovitz, and W. A. De Brouwer. Compiled AI: Deterministic Code Generation for LLM-Based Workflow Automation. *arXiv:2604.05150*, 2026.
- [11] M. Konstantinou, F. Tambon, and M. Papadakis. On the risk of coding before testing: An empirical study on LLM-based test generation workflow. *arXiv:2607.05139*, 2026.
- [12] J. Yates. *Control through Communication: The Rise of System in American Management*. Johns Hopkins University Press, 1989.
- [13] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the Middle: How Language Models Use Long Contexts. *TACL* 12:157–173, 2024. *arXiv:2307.03172*.

- [14] A. Modarressi, H. Deilamsalehy, F. Deroncourt, T. Bui, R. A. Rossi, S. Yoon, and H. Schütze. NoLiMa: Long-Context Evaluation Beyond Literal Matching. In *ICML*, 2025. arXiv:2502.05167.
- [15] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou. Large Language Models as Tool Makers. *arXiv:2305.17126*, 2023.
- [16] S. El Yadouni and G. Li. TraceCompiler: Skill-Guided Mining and Compilation of LLM Agent Traces into Mostly Deterministic Workflows. *arXiv:2608.02680*, 2026.
- [17] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *arXiv:2310.03714*, 2023.